

FORMATION SPRING

Programmation réactive avec Spring WebFlux

De Spring MVC au non-bloquant : concepts, Reactor, WebFlux et bonnes pratiques

Pour développeurs Spring MVC • Niveau intermédiaire



Objectifs de la formation

- 1 Comprendre les limites du modèle synchrone « thread-per-request »
- 2 Assimiler les concepts de Reactive Streams et de Project Reactor (Mono / Flux)
- 3 Écrire des contrôleurs et endpoints réactifs avec Spring WebFlux
- 4 Consommer des services externes de façon non bloquante avec WebClient
- 5 Savoir tester du code réactif et choisir MVC ou WebFlux à bon escient

Le modèle Spring MVC : un thread par requête

Pool de threads Tomcat (taille fixe)



En rouge : threads bloqués en attente d'E/S (base de données, appel HTTP...)

- Chaque requête HTTP occupe un thread pendant toute sa durée
- Un appel bloquant (JDBC, RestTemplate...) immobilise ce thread
- Sous forte charge : le pool s'épuise → requêtes en file d'attente
- Scalabilité limitée par le nombre de threads, pas par le CPU

Qu'est-ce que la programmation réactive ?

Un style de programmation asynchrone, orienté flux de données (« streams ») et non bloquant, où le consommateur peut réguler le débit qu'il reçoit (backpressure).

1

Responsive

Le système répond rapidement, même sous charge

2

Resilient

Il reste réactif face aux pannes, via isolation

3

Elastic

Il s'adapte à la charge en allouant les ressources utiles

4

Message-Driven

Les composants communiquent par messages asynchrones

Reactive Streams : le contrat commun

Une spécification (intégrée à Java 9 sous `java.util.concurrent.Flow`) que Project Reactor, RxJava et Akka implémentent tous.

Publisher<T>

Émet une séquence d'éléments T à ses abonnés

Subscriber<T>

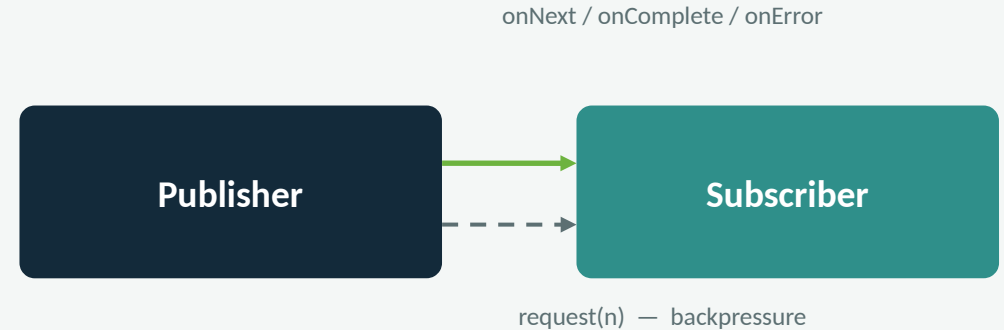
Reçoit les éléments, les erreurs et le signal de complétion

Subscription

Relie un Publisher à un Subscriber ; permet de demander N éléments

Processor<T, R>

À la fois Subscriber et Publisher (étape de transformation)

**Backpressure**

Le Subscriber contrôle le débit en demandant explicitement combien d'éléments il souhaite recevoir — il ne peut pas être submergé par un Publisher trop rapide.

Mono et Flux : les deux types réactifs

Mono<T>

0 ou 1 élément

Équivalent conceptuel

≈ Optional<T> / CompletableFuture<T>, en asynchrone

```
Mono<User> user = userRepo.findById(id);
```

Flux<T>

0 à N éléments (potentiellement infini)

Équivalent conceptuel

≈ List<T> / Stream<T>, en asynchrone

```
Flux<User> users = userRepo.findAll();
```

Créer et manipuler un Flux

Un Flux ne fait rien tant que personne n'y est abonné : il est paresseux (« lazy »).

```
Flux<String> flux = Flux.just("a", "b", "c")
    .map(String::toUpperCase)
    .filter(s -> !s.equals("B"))
    .doOnNext(s -> log.info("reçu: {}", s));

// Rien ne se passe tant qu'on ne souscrit pas :
flux.subscribe();
// → affiche : reçu: A    reçu: C
```

À retenir

- Les opérateurs (map, filter...) construisent un pipeline, ils ne l'exécutent pas.
- Rien ne s'exécute avant subscribe() — c'est la paresse (laziness).
- Dans un contrôleur WebFlux, c'est Spring qui souscrit à votre place.

Les opérateurs à connaître en premier

Opérateur	Rôle	Analogie Java/MVC
map	Transforme chaque élément (synchrone)	<i>Stream.map</i>
flatMap	Transforme en un nouveau Publisher et l'aplatit (async)	<i>chaîner deux appels bloquants</i>
filter	Ne garde que les éléments respectant une condition	<i>Stream.filter</i>
zip	Combine plusieurs flux élément par élément	<i>attendre 2 résultats en parallèle</i>
merge / concat	Fusionne plusieurs flux (parallèle / séquentiel)	<i>agréger plusieurs sources</i>
subscribe	Déclenche l'exécution et consomme les éléments	<i>point d'entrée du flux</i>

Piège n° 1 : ne jamais bloquer un flux réactif

Un seul appel bloquant dans une chaîne réactive peut geler tout un event loop Netty et donc toutes les requêtes qu'il sert.

✗ À éviter

```
Mono<User> user = webClient.get()
    .retrieve()
    .bodyToMono(User.class)
    .block(); // ⚠ bloque le thread

String data = jdbcTemplate
    .queryForObject(...); // ⚠ JDBC bloquant
```

✓ À privilégier

```
Mono<User> user = webClient.get()
    .retrieve()
    .bodyToMono(User.class);
// composer, ne pas bloquer
return user.flatMap(u ->
    reactiveRepo.save(u));
```

Gestion des erreurs en réactif

Les erreurs sont des signaux du flux (onError) — elles se gèrent avec des opérateurs dédiés, pas avec try/catch.

```
webClient.get().uri(url)
    .retrieve().bodyToMono(Order.class)
    .timeout(Duration.ofSeconds(3))
    .retry(2)
    .onErrorResume(TimeoutException.class,
        e -> Mono.just(Order.fallback()))
    .onErrorReturn(Order.empty());
```

Opérateurs utiles

onErrorReturn — valeur de repli statique

onErrorResume — bascule vers un autre Publisher

retry(n) / retryWhen — nouvelle tentative

timeout(d) — borne la durée d'attente

Sous le capot : Tomcat vs Netty

Spring MVC — Tomcat

- Pool de threads borné
- 1 thread = 1 requête, du début à la fin
- Modèle synchrone / bloquant
- Simple à raisonner et déboguer

Spring WebFlux — Netty

- Petit nombre de threads (event loop)
- 1 thread gère de nombreuses requêtes
- Modèle asynchrone / non bloquant
- Débit élevé si le code ne bloque jamais

Des contrôleurs annotés très proches de MVC

Même annotations (@RestController, @GetMapping) — seul le type de retour change.

Spring MVC

```
@GetMapping("/users/{id}")
public User getUser(
    @PathVariable String id) {
    return userService
        .findById(id); // bloquant
}
```

Spring WebFlux

```
@GetMapping("/users/{id}")
public Mono<User> getUser(
    @PathVariable String id) {
    return userService
        .findById(id); // non bloquant
}
```

Alternative : les endpoints fonctionnels

WebFlux propose aussi un style fonctionnel : routes déclarées par du code, sans annotations.

```
RouterFunction<ServerResponse> routes() {  
    return RouterFunctions.route()  
        .GET("/users/{id}", this::getUser)  
        .POST("/users", this::createUser)  
        .build();  
}  
  
Mono<ServerResponse> getUser(ServerRequest req) {  
    return userService.findById(req.pathVariable("id"))  
        .flatMap(ServerResponse.ok()::bodyValue);  
}
```

Quand l'utiliser ?

- Routage explicite et testable comme du code normal
- Pratique pour des APIs très dynamiques ou modulaires
- Coexiste sans problème avec des contrôleurs annotés

WebClient : le remplaçant réactif de RestTemplate

RestTemplate est en mode maintenance depuis Spring 5 — WebClient est non bloquant et supporté aussi bien en MVC qu'en WebFlux.

```
WebClient client = WebClient.builder()
    .baseUrl("https://api.exemple.com")
    .build();

Mono<Order> order = client.get()
    .uri("/orders/{id}", orderId)
    .retrieve()
    .bodyToMono(Order.class);
```

Utilisable partout

Dans un contrôleur MVC, on peut renvoyer le résultat avec `.block()` — à faire avec parcimonie, uniquement en bordure d'application.

Dans WebFlux, on chaîne directement sans bloquer.

Accès aux données réactif : R2DBC

JDBC est bloquant par nature. R2DBC (Reactive Relational Database Connectivity) apporte l'accès non bloquant aux bases relationnelles.

	JDBC / JPA	R2DBC
Modèle	Bloquant	Non bloquant
Repository	CrudRepository	ReactiveCrudRepository
Type de retour	T / List<T>	Mono<T> / Flux<T>
Pool de connexions	HikariCP	r2dbc-pool

```
interface UserRepository extends  
    ReactiveCrudRepository<User, String> { }
```

Tester du code réactif avec StepVerifier

reactor-test fournit StepVerifier pour vérifier, étape par étape, ce qu'un Publisher émet.

```
@Test
void devraitFiltrerLesUtilisateursInactifs() {
    Flux<User> result = service.findActiveUsers();

    StepVerifier.create(result)
        .expectNextMatches(User::isActive)
        .expectNextCount(2)
        .verifyComplete();
}
```

Points clés

- Aucun `.block()` nécessaire dans les tests
- Peut simuler le temps (virtual time) pour tester des `delay()/timeout()`
- S'intègre à JUnit 5 comme un test classique

Bonnes pratiques & pièges à éviter

✓ À faire

- Composer avec map/flatMap plutôt que bloquer
- Propager Mono/Flux jusqu'au bout de la chaîne
- Gérer les erreurs avec les opérateurs dédiés
- Utiliser WebClient pour tout appel externe

✗ À éviter

- Appeler .block() dans un contrôleur WebFlux
- Mélanger JDBC bloquant et pipeline réactif
- Faire du CPU lourd sur l'event loop sans le isoler
- Ignorer la gestion d'erreurs (« ça marchera bien »)

WebFlux ou MVC ? Grille de décision

Critère	Spring MVC	Spring WebFlux
Charge principalement CPU	✓ Recommandé	Pas d'avantage net
Beaucoup d'appels I/O concurrents	Limité par les threads	✓ Recommandé
Équipe déjà à l'aise en réactif	Non requis	✓ Nécessaire
Dépendances bloquantes (JDBC, libs)	✓ Naturel	Risque de blocage
Streaming de données / SSE	Possible mais limité	✓ Idéal
Simplicité de debug	✓ Plus simple	Courbe d'apprentissage

Ressources pour aller plus loin



Documentation officielle

docs.spring.io/spring-framework/reference/web/webflux.html



Project Reactor

projectreactor.io – guide de référence et « Reactor by Example »



Reactive Streams

reactive-streams.org – la spécification complète



Test

projectreactor.io/docs/test/release/reference – StepVerifier



Livre

"Reactive Spring" de Josh Long

MERCI

Des questions ?

Programmation réactive avec Spring WebFlux — Formation Spring

